



University of Glasgow

Course	CSC3106 Cyber Security
Title	Data-Driven Security Analysis and Defensive Response
Team	X
Assigned Extract	4_auth.log
Chosen Asset	backup01, specifically its SSH access surface and the privileged backup account.

Our members:

Name	GUID	SIT ID	Task Distribution
William Oon Zhen Wei	3070657O	2401532	Equal
Kieran Sim	3070564S	2403348	Equal
Tan Bing Kun Terence	3070672T	2401883	Equal
Dui Ru-En Joshua	3070683D	2402201	Equal

Part 1: Data-Driven Authentication Log Analysis

1. Security Question and Scope

What phases of a cyberattack directed at `backup01` can be reconstructed through SSH authentication and session data within `4_auth.log`, and which defensive measures are most critical for securing privileged accounts?

The scope focuses on host `backup01`, centering on its SSH service and the operational `backup` user. This account was subjected to intense targeting and demonstrated an accepted login during an attack wave, followed by immediate `sudo` activity as `root`. Our findings are based on a log period from `Jul 06` to `Jul 12`. All quantitative data and visualizations were derived via a custom Python script (`analysis.py`). The report emphasizes the reconstruction of sequential attack stages, distinguishing empirical log data from security interpretations and highlighting inherent visibility gaps. As noted in Section 6, `auth.log` provides an incomplete picture of post-compromise actions such as data exfiltration or system tampering.

2. Method Summary

`analysis.py` decomposes log entries into standard syslog components before categorizing `sshd` and `sudo` messages into discrete security events. The script extracts critical metadata, including user identities, origin IPs, and varied authentication outcomes. It specifically identifies session transitions and privileged command executions while maintaining visibility into parsing coverage via separate counts for ambiguous data (refer to Appendix D). Because the source file lacks strict chronological ordering, the tool performs a global sort by timestamp to ensure accurate timeline reconstruction.

3. Evidence Overview

The extract contains **12,000 lines**, all for `backup01`.

Event type	Count
Failed password attempts	5,176
Accepted password logins	1,859
Invalid-user attempts	751
Maximum authentication attempts exceeded	640
Sudo commands	255

Table 1: Headline Counts

The high failed-password volume indicates broad SSH credential-guessing. Four sources dominate (`203.0.113.88`, `198.51.100.88`, `203.0.113.89`, `198.51.100.89`); full counts are in Appendix B. The username targeting (`root`, `admin`, `deploy`, `postgres`, `oracle`) and sustained volume are consistent with **automated, opportunistic SSH scanning** rather than a bespoke targeted campaign, activity that is common internet background noise. The volume is therefore treated as context, not a risk in itself. What elevates it is that only `203.0.113.89` progresses from failed attempts to a successful login: an exposed password-authenticated privileged account was compromised by otherwise-commodity guessing. This makes the *successful access*, not the raw attempt count, the priority finding, and it is why the standing exposure warrants a defensive response even though the guessing itself is routine.

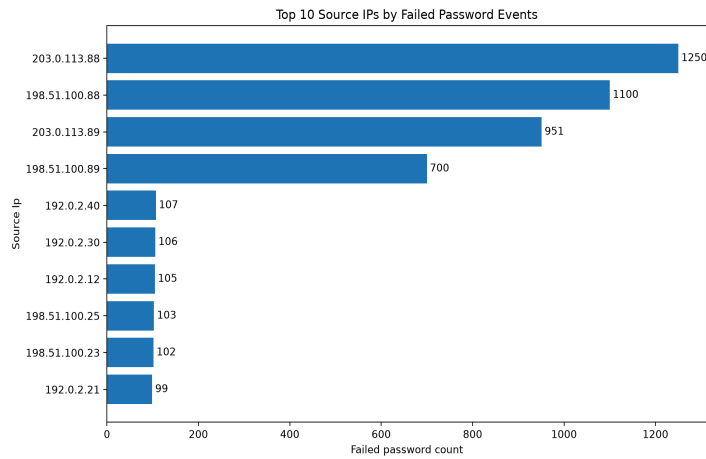


Figure 1: Top source IPs by failed password attempts

4. Attacker Lifecycle: the 203.0.113.89 sequence

The strongest finding is a single-source lifecycle on Jul 11. 203.0.113.89 produced sustained failed attempts and probing, then a successful backup login, an SSH session, and immediate root-level sudo activity. Mapped to MITRE ATT&CK:

Evidence (from 203.0.113.89)	Lifecycle stage	ATT&CK
03:07:30 failed login, invalid user oracle; common/service usernames probed	Account probing	T1110.001; consistent with T1087
03:07:32 failed root login	Privileged-account targeting	T1110.001
03:08:19 Accepted password for backup	Valid-account access over SSH	T1078, T1021.004
03:08:19 session opened for user backup	Session establishment	T1021.004
03:08:19 backup runs sudo ... journalctl -n 200, systemctl status cron (as root)	Privileged post-login activity	T1548.003

Table 2: Critical Jul 11 authentication-to-sudo sequence

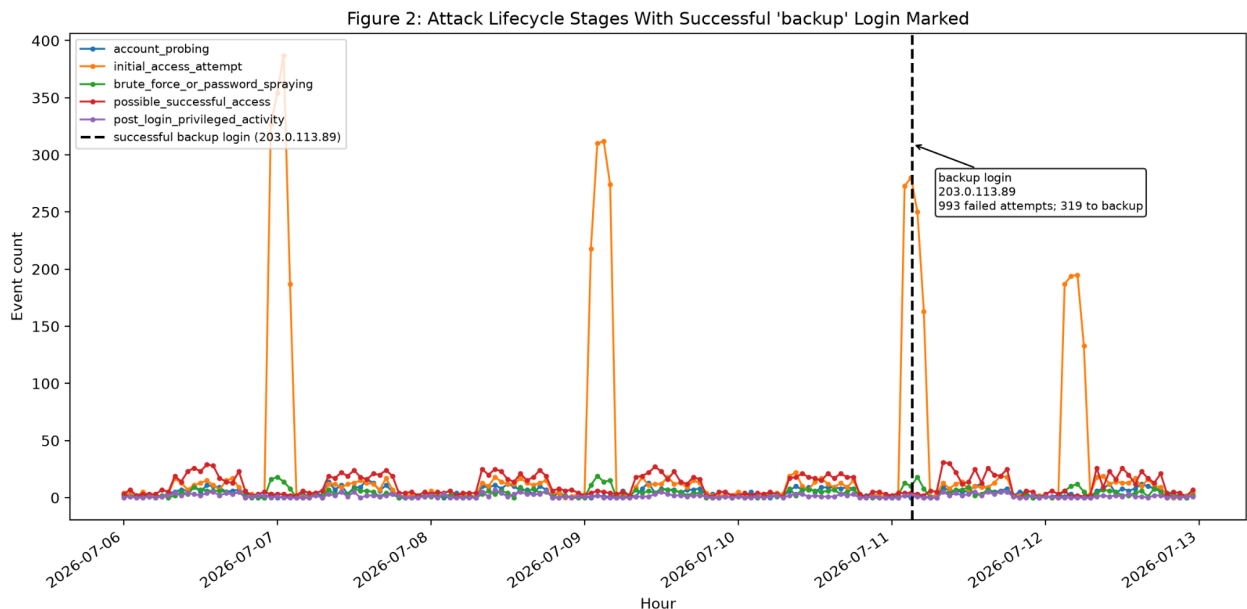


Figure 2 — Failed attempts over time with the successful **backup** login marked

The sequence maps most defensibly to **T1110.001** → **T1078** → **T1021.004** → **T1548.003**: password guessing that reaches a valid account over SSH, whose `sudo` rights expose a path to root. The two `sudo` commands (`journalctl`, `systemctl status cron`) are individually legitimate administrative actions, so they are interpreted as **possible privileged reconnaissance** immediately after a suspicious login — not confirmed malicious action. Same-second (**03:08:19**) timestamps reflect syslog's one-second precision and do not prove millisecond ordering, but the rapid login-to-`sudo` transition is more consistent with scripted or prepared activity than routine manual administration. A short raw excerpt is in Appendix C.

Temporal pattern. The brute-force activity is concentrated outside working hours: roughly 70% of failed logins (3,643 of 5,176) fall between 00:00 and 05:59, with a further spike around 23:00. Legitimate accepted logins show the opposite profile, clustering in business hours (08:00–18:00). In the overnight window there are 3,643 failed against only 149 accepted logins, whereas daytime is close to balanced (1,533 failed vs 1,710 accepted). The successful **backup** login at **03:08:19** therefore occurs at the peak of the attack window and well outside the host's normal login baseline, which strengthens the case that it is attacker-driven rather than routine administration (see Appendix B, Figure 3). This off-hours concentration also motivates a time-aware detection control in Part 2. Host-local time is assumed; no timezone is recorded in the extract.

5. Risk Prioritisation

Asset: **backup01** — its SSH access surface, the **backup** account, and the `sudo`→root path. The three prioritised risks trace the observed attack path (access → escalation → impact)

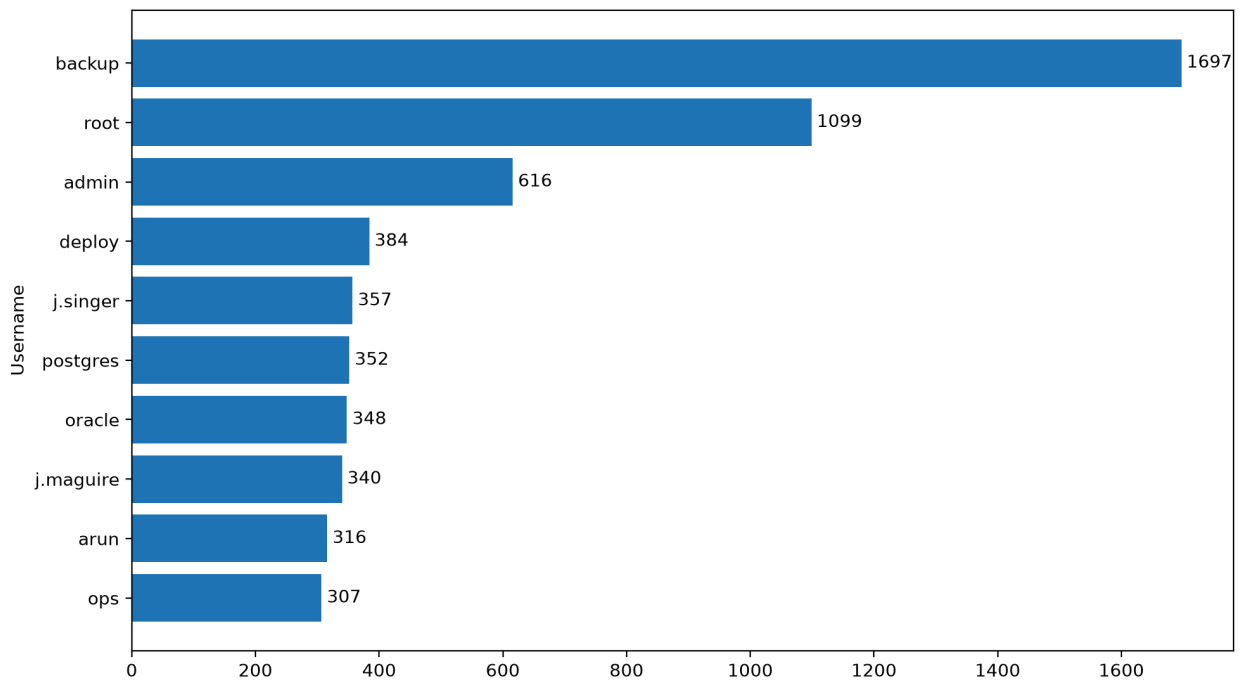


Figure 3 - Top 10 targeted usernames

ID	Risk (consequence to asset)	Likelihood	Impact	Evidence	Uncertainty
R1	Unauthorised privileged access via compromised <code>backup</code> SSH credentials	High	High	951 fails from <code>203.0.113.89</code> , then <code>Accepted password for backup</code> , then immediate <code>sudo</code>	Cannot confirm the login was unauthorised vs. legitimate reuse
R2	Escalation to root through <code>backup</code> 's <code>sudo</code> rights	High	Critical	<code>backup</code> runs <code>sudo ... USER=root</code> seconds after login	No destructive root action visible in the extract
R3	Loss of backup-data integrity/availability on <code>backup01</code>	Medium	Critical	Root path on a dedicated backup host	Not directly observed in <code>auth.log</code>

R1 is the top priority: it combines attempted access, apparent success, and privileged activity in one source. Secondary risks — SSH-service degradation from high-volume brute force, and exposure of other `sudo`-capable accounts (`ops`, `arun`, `j.singer`, `j.maguire`) — are noted but not prioritised here. The targeting distribution also supports prioritising the `backup` account. As shown in Figure 3, `backup` received the highest number of failed password and maximum-authentication events among extracted usernames.

6. Initial Response and Limitations

The immediate response should be to verify whether the backup login from 203.0.113.89 was authorised, rotate or disable the backup account credentials until confirmed, preserve relevant logs, and review sudo, cron, journal, shell history, and audit evidence. High-volume SSH sources should also be blocked or rate-limited.

This evidence is limited to auth.log. It shows authentication, session, and sudo activity, but not every shell command. Therefore, the extract supports likely brute-force activity, possible successful credential compromise, and privileged reconnaissance, but it does not prove botnet activity, malware execution, persistence, exfiltration, log deletion, or destructive tampering. The IP ranges are documentation-style ranges, so no real-world attribution or geolocation should be inferred.

Part 2: Technical Defensive Response

Selected Risk

R1 — Unauthorised privileged access to backup01 via the compromised backup account, reached through opportunistic SSH brute force from 203.0.113.89 and extended to root through the account's sudo rights.

This is the priority because it is the one point where the evidence moves beyond failed attempts: a suspicious source guessed passwords, obtained an accepted login to a privileged account, opened a session, and immediately ran sudo as root. The defence therefore targets the full path rather than brute force alone:

```
SSH brute force → successful backup login → active session → sudo as root → system tampering
(prevent)          (prevent + detect)      (contain)          (contain)          (contain)
```

The response is layered so that no single control is a single point of failure: **prevention** to stop guessing succeeding, **containment** to limit damage if a credential is still compromised, and **detection** to catch a success the preventive layer misses. This directly addresses the ATT&CK path identified in Part 1 (T1110.001 → T1078 → T1021.004 → T1548.003).

Defensive Response

The response combines three layers, each mapped to the attack path and the Part 1 evidence.

Attack stage (ATT&CK)	Control	Effect
Password guessing (T1110.001)	Enforce key-only SSH (PasswordAuthentication no); rate-limit/fail2ban on repeated failures; PermitRootLogin no	Guessing a password no longer grants access; automated brute force cannot succeed by default
Valid-account access over SSH (T1078, T1021.004)	Restrict the backup key by source (from=) and forced	A stolen password is useless; even a stolen key only runs

Attack stage (ATT&CK)	Control	Effect
	command; disable interactive shell if not required	approved backup operations from approved hosts
Privilege escalation via sudo (T1548.003)	Least-privilege sudoers for <code>backup</code> — only the specific backup commands, no shell/ <code>su/sudo</code>	A compromised <code>backup</code> context can no longer obtain a general root shell
Post-compromise tampering	SELinux (enforcing) confining the backup domain to only its required files	Even with residual privilege, the account cannot read/modify critical system files
Evidence gaps after login	Central log forwarding + sudo/audit logging	Preserves evidence beyond the local <code>auth.log</code> for investigation

Prevention: remove the exposed password surface

The root cause of R1 is password authentication on a privileged, internet-facing account. Disabling `PasswordAuthentication` and requiring SSH keys removes the entire brute-force success path; `fail2ban`-style rate-limiting reduces the noise and load from the ongoing automated scanning identified in Part 1. Denying direct root SSH login closes the most-targeted account (`root`, 1,048 failed attempts) entirely.

Containment: least privilege for the `backup` account

Because `backup` is an operational/service account, broad interactive `sudo` access is unnecessary risk. Restricting its sudoers entry to the exact commands its backup jobs require — and explicitly denying shell, `su`, and `sudo` — prevents a compromised credential from being escalated to a general root shell (the T1548.003 step observed at 03:08:19).

Containment: SELinux to protect critical files

Discretionary permissions and sudo rules can still be bypassed if an attacker reaches root. **SELinux in enforcing mode** adds mandatory access control: the backup service runs in a confined domain that is permitted to touch only its own data paths. Even a compromised `backup` context is denied write access to sensitive system files such as `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`, cron directories, and SSH host keys, because those files carry protected types the backup domain has no policy allowance for. This limits both privilege escalation (e.g. editing `/etc/passwd` or `sudoers`) and anti-forensic tampering (e.g. altering logs or cron persistence), turning a full host compromise into a contained one.

Detection: catching a success the preventive layer misses

If a credential is still compromised, the failure-then-success transition should raise a high-priority alert. A naive "failed then accepted from the same IP" rule is too broad — in

this extract **13 source IPs match that pattern, but 12 are legitimate users** who occasionally mistype before authenticating (13–28 failures, with successes outnumbering failures). Selectivity therefore comes from tuning to volume and ratio, not the raw pattern.

Alert when, from a single source IP within a defined window:

1. failed SSH attempts exceed a threshold set above the legitimate baseline — legitimate sources here peaked at ~28 failures, so ≥ 50 cleanly separates them (203.0.113.89 produced 311);
2. **and** the failure-to-success ratio is heavily skewed (legitimate $\approx 1:1$; 203.0.113.89 = 311:1);
3. **and** the same IP then obtains an accepted login to the privileged **backup** account (the only account that successfully authenticates in this log — **root**, **admin**, and **deploy** appear only as failed/invalid targets);
4. weighted higher when the success and follow-on **sudo** fall outside business hours (the Part 1 attack window was 00:00–05:59; the compromise was at 03:08).

Applied to the extract, this rule flags 203.0.113.89 and suppresses the 12 legitimate mixed-outcome sources, giving a high-confidence signal that combines volume, ratio, target sensitivity, and timing.

Effectiveness, Limitations, and Residual Risk

How the response reduces risk. Key-only SSH and root-login denial *reduce likelihood* by removing the password-guessing success path entirely. Least- privilege sudo and SELinux confinement *reduce impact* by preventing a compromised **backup** credential from becoming general root control or tampering with critical files. The tuned correlation rule and central logging *improve detection and investigation* for any success the preventive layer misses.

Limitations and trade-offs.

- **Detection selectivity depends on tuning.** Thresholds set too low re-introduce the 12 legitimate false positives; too high may miss a low-and-slow attacker. The rule mitigates single-source brute force but not a distributed campaign spread across many IPs.
- **Rate-limiting/lockout can cause denial of service.** Aggressive **fail2ban** thresholds could block legitimate operations or be abused to lock out accounts; thresholds must be tested so scheduled backup jobs are not interrupted.
- **Key management overhead.** Key-only SSH shifts risk to key storage and rotation; a stolen key still authenticates unless source- and command-restricted.
- **SELinux is not absolute.** Policy authoring and maintenance are non-trivial, mislabelling can break services, and an attacker who reaches an unconfined admin role could still relabel or disable enforcement. It raises the cost of compromise rather than eliminating it.

Residual risk. The controls improve prevention, containment, detection, and investigation, but they do not retrospectively prove what happened after the **Jul 11** login.

Confirming or clearing that event still requires the wider evidence review recommended in Part 1 (shell history, full journal, audit logs, cron and file-integrity checks).

Appendices (Supporting Material)

Appendix B — Full Evidence Tables

Top source IPs by failed authentication attempts

Source IP	Failed	Note
203.0.113.88	1,250	High volume; no accepted login
198.51.100.88	1,100	High volume
203.0.113.89	951	High volume + one successful backup login
198.51.100.89	700	High volume

Most-targeted usernames (failed attempts)

Username	Failed	Relevance
backup	1,552	Sensitive; successful login from 203.0.113.89
root	1,048	Direct privileged targeting
admin	589	Common admin target
deploy	366	Operational account
postgres	336	Service/DB account
oracle	331	Service/DB account

Failed vs accepted logins by time window

Window	Failed	Accepted	Note
Night (00:00–05:59)	3,643	149	Brute-force concentrated here (~70% of all failures)
Day (06:00–23:59)	1,533	1,710	Legitimate activity dominates business hours

Peak failed-login hours are 02:00–04:00; a secondary spike occurs at 23:00. Peak accepted-login hours are 08:00–18:00. The successful `backup` login (`Jul 11 03:08:19`) falls inside the peak attack window.

Figure 3 — Failed vs accepted authentication attempts by hour of day [`insert output/attempts_by_hour.png`]

Appendix C — Raw Log Excerpt (203.0.113.89, Jul 11)

Jul 11 03:07:30 backup01 sshd[1675]: Failed password for invalid user oracle from 203.0.113.89 port 56854 ssh2

Jul 11 03:07:32 backup01 sshd[1447]: Failed password for root from 203.0.113.89 port 40703 ssh2

Jul 11 03:08:19 backup01 sshd[2164]: Accepted password for backup from 203.0.113.89 port 59595 ssh2

Jul 11 03:08:19 backup01 sshd[7367]: pam_unix(sshd:session): session opened for user backup by (uid=0)

Jul 11 03:08:19 backup01 sudo: backup : TTY=pts/3 ; PWD=/home/backup ; USER=root ; COMMAND=/usr/bin/journalctl -n 200

Jul 11 03:08:19 backup01 sudo: backup : TTY=pts/3 ; PWD=/home/backup ; USER=root ; COMMAND=/usr/bin/systemctl status cron

Appendix D — Parsing Assumptions and Coverage

- Format assumed: `Mon DD HH:MM:SS host process[pid]: message`; no year in timestamps.
- Username handling covers both `Failed password for USER` and `Failed password for invalid user USER`.
- Non-standard lines (e.g. `Connection closed by authenticating user ..., reverse mapping ... POSSIBLE BREAK-IN ATTEMPT`) are classified or counted as unmatched, not silently dropped.
- Unmatched-line count: [`insert from analysis.py`].

Appendix E — AI-Use Declaration and Workload

- AI-use declaration: [`state which AI tools were used, for what, and how outputs were checked/verified – or omit this appendix to declare no AI tools were used`].
- Workload record: "All members contributed equally," or a short per-member breakdown.